

1. Python is a dynamically typed language
2. Variables are represented using snake case, ex: friend_age and not friendAge (as in JS)
3. Variables are printed using print()
4. Constants are represented using All Caps lock for variables, value of these variables can be changed but it is not recommended, ex: PI=3.14, EARTH_DIAMETER=151
5. Mathematics operations follow BODMAS rules
6. Integers and floating point are 2 types of numbers - Integers are whole numbers, ex: 3, 4, 4.15 - these are the 2 types of numbers
7. Variable can be re-assigned since its value can vary - by the very meaning of the term variable
8. Division can be of 2 types:
 - a. float_division = 8/3 = 2.66666
 - b. integer_division = 8 // 3 = 2
9. Modulo operator = % => 4%2 = 0, 5%2 = 1
10. Data Types in Python:
 - a. Numbers
 - b. Text
11. my_string = "Hello! World", my_string = 'Hello! World', we can use Single/Double Quotes to represent strings
12. my_string = 'Hello! "This is an amazing day!" yesterday', my_string = "Hello! "This is an amazing day!" yesterday"
13. multiline = """"Hello, world.

My name is Ravi Nayak. Welcome to my program.

""""
14. Multiline strings are represented using TRIPLE double quotation marks which preserve the new lines in text/string
15. 2 data types can be added as long as they have the same data type, for ex: strings can be added
 - a. str1 = "Jose"
 - b. str2 = "Welcome here"
 - c. print(str1 + str2)
16. 2 data types cannot be added if they have different data types, for ex: a string and a number cannot be added, it throws an error
 - a. Convert a number to string
 - b. age = 34, age_str = "age:"
 - c. print(age_str + str(age))
17. f strings are an alternate to converting integers to strings for display purposes
 - a. age = 34
 - b. print(f'You are {age}')
18. f strings result in static allocation of variable values
 - a. name = "Jose"
 - b. greeting = f'How are you, {name}?'
 - c. print(greeting)
 - d. name = "Bob"
 - e. print(greeting)
 - f. Although we change name from "Jose" to "Bob", the value of greeting remains same

- => How are you, Jose?
- g. This is because greeting is evaluated at the beginning when name is assigned a value of "Jose"
19. An alternate to f strings are to define strings using empty curly braces and format function, which can be used to dynamically assign value of a variable
- name = "Jose"
 - final_greeting = "How are you, {}?"
 - jose_greeting = final_greeting.format(name)
 - print(jose_greeting)
 - name = "Bob"
 - bob_greeting = final_greeting.format(name)
 - print(bob_greeting)
 - final_greeting_1 = "How are you, {name}?"
 - jose_greeting = final_greeting.format(name=name)
20. f strings are the weapon of choice, and used mostly in Python due to their simplicity
21. When we want to use templates, we would use the option defined in 19
22. Input from User to get their information:
- your_name = "Rolf"
 - user_name = input('Enter your name:')
 - print(f'Hello! {user_name}, My name is {your_name}')
 - Any information entered by user is always a string
 - age = input('Enter your age:')
 - age_num = int(age)
 - print(f'You have lived for {age * 12} months') => You have lived for 33333333333 months
 - A string multiplied by another string results in concatenation
 - "12" * "3" = 3 repeated 12 times
 - print(f'You have lived for {age_num * 12} months') => You have lived for 36 months
23. truthy = True
24. falsey = False
25. Short Circuit operator (for AND) in Python is represented by "and" (NOT &&)
26. Short Circuit operator (for OR) in Python is represented by "or" (NOT ||)
27. bool function if called on
- 0
 - "" (Empty String)
 - [] (Empty List) results in False
28. bool(0) = bool("") = bool([]) = False
29. Lists can contain data which is NOT Homogenous but it is generally recommended that we keep Homogeneous data in the list
- friends = ['Rolf', 'Anne', 'Jose']
 - friends_ages = [['Rolf', 24], ['Anne', 30], ['Jose', 35]]
 - print(friends[0])
 - print(friends[0][0])
 - If a List is long, it is generally recommended to spread the list across several lines with indentation
 - friends = [['Rolf', 24], ['Jose', 31], ['Anne', 35].]
 - friends.remove(['Anne', 35])
30. Data Structures in Python:
- List: [], Mutable, elements can be added and removed, [] used, think arrays, Python 3.7 + => Order guaranteed => Elements maintain the order in which they were inserted
 - Tuple: (), Immutable, elements cannot be added or removed, () used, Python 3.7 + => Order guaranteed (elements retain their sequence) => Tuples were always

ordered

- c. Set: {}, Mutable, cannot contain duplicates, effective for difference, intersection, and union, order not guaranteed => Sets do not maintain order, order in which elements were inserted can change unexpectedly
- d. Dictionary: {}, Mutable, cannot contain duplicate keys, think hash, Python 3.7 + => Order guaranteed => Guarantees Insertion Order

31. Tuples:

- a. It is recommended to use ()
- b. In certain cases like usage within Lists, Python would not know a tuple if () were not used
- c. short_tuple = "Rolf", "Bob"
- d. clearer_tuple = ("Rolf", "Bob")
- e. list with tuples = [("Rolf", "Bob")]
- f. ["Rolf", "Bob"] is not a list with Tuples
- g. friends = ("Rolf", "Bob"), print(friends) => Here friends is immutable, implying that we cannot add/remove from a tuple => friends.add / friends.remove methods are not supported => friends = friends + ("Jordan") => the original tuple is retained, we assign a new tuple to friends => A variable representing a tuple can be re-assigned a new tuple, but tuple defined originally cannot be mutated

32. Sets:

- a. art_friends = { 'Rolf', 'Jacob', 'Jen' }
- b. science_friends = { 'Jen', 'King' }
- c. art_friends_but_not_science_friends = art_friends - science_friends = art_friends.difference(science_friends) => Elements which are present in art_friends set but not in science_friends set => { 'Rolf', 'Jacob' }
- d. art_friends.symmetric_difference(science_friends) = science_friends.symmetric_difference(art_friends) => { 'King', 'Rolf', 'Jacob' } => Includes all elements of art_friends and science_friends without the intersection of those sets
- e. art_friends.union(science_friends) = science_friends.union(art_friends) => All elements of two sets (art_friends, science_friends) with the intersection elements listed only once => { 'Jacob', 'Jen', 'King', 'Rolf' }
- f. intersection_art_science_friends = art_friends.intersection(science_friends) => { 'Jen' }
- g. Add/Remove elements to/from a set: => Add element to a set => art_friends.add('Ron') => { 'Rolf', 'Jacob', 'Jen', 'Ron' } => Remove element from a set => art_friends.remove('Ron') => { 'Rolf', 'Jacob', 'Jen' }

33. Dictionary: Hash - Key/Value Pairs => friend_ages = { 'Rolf': 24, 'Adam': 30, 'Anne': 27 } => print(friend_ages) => Access value of a specific key = print(friend_ages['Rolf']) => Set existing key to a new value => friend_ages['Rolf'] = 35 => Add new key/value to dictionary => friend_ages['Wolf'] = 45 => dict method: Can be used to create a new dictionary from a list of tuples => friends = [('Rolf', 24), ('Anne', 30), ('Jen', 35)] => dict(friends) => friends = ({ 'name': 'John', age: 35 }, { 'name': 'Wolf', age: 40 }) [A tuple with dictionaries]

34. grades = [80, 75, 90, 100] => total = sum(grades) => length = len(grades)

35. friends = ['Ann', 'John', 'Wolf'] => joined_str = ','.join(friends)